

Documentação Técnica

Módulo Analisador

Sistema SIVIRA
Sistema Inteligente para Visualização e
Replanejamento de Atividades

6 de outubro de 2025

Resumo Executivo

Este documento apresenta a documentação técnica completa do módulo **analisador**, responsável pela análise de logs de produção, detecção de atividades duplicadas e cálculo de reagendamentos utilizando backward scheduling.

Conteúdo

1	Visão Geral	3
1.1	Objetivo	3
1.2	Arquitetura	3
2	analisador_pedidos.py	4
2.1	Descrição	4
2.2	Classe AnalisadorPedidos	4
2.2.1	Atributos	4
2.2.2	Métodos Principais	4
2.3	Formato dos Logs	5
3	calculador_reagendamento.py	6
3.1	Descrição	6
3.2	Classe CalculadorReagendamento	6
3.2.1	Atributos	6
3.2.2	Métodos Principais	6
3.3	Algoritmo de Backward Scheduling	7
4	executar_reagendamento.py	8
4.1	Descrição	8
4.2	Funções Principais	8
4.3	Fluxo de Execução	8
5	Fluxo de Dados	9
5.1	Diagrama de Fluxo	9
5.2	Estrutura de Dados	9
5.2.1	Dicionário de Pedidos	9
5.2.2	Dicionário de Duplicatas	9
6	Exemplo de Uso	10
6.1	Uso Programático	10
6.2	Uso via Interface	10
7	Padrões e Convenções	11
7.1	Documentação	11
7.2	Nomenclatura	11
7.3	Estrutura de Código	11
8	Manutenção e Extensão	12
8.1	Adicionar Novos Formatos de Log	12
8.2	Algoritmos Alternativos	12
8.3	Validações Adicionais	12
9	Dependências	12
9.1	Bibliotecas Python	12
9.2	Módulos Internos	12
10	Conclusão	12

1 Visão Geral

O módulo **analisador** é composto por três componentes principais que trabalham em conjunto para analisar logs de equipamentos, identificar conflitos de recursos compartilhados e calcular reagendamentos necessários.

1.1 Objetivo

O objetivo principal do módulo é:

- Carregar e processar logs de produção de equipamentos
- Identificar atividades compartilhadas entre diferentes pedidos
- Calcular reagendamentos utilizando backward scheduling
- Fornecer interface interativa para seleção e análise de conflitos

1.2 Arquitetura

O módulo está organizado em três arquivos Python:

1. `analisador_pedidos.py` - Análise e indexação de logs
2. `calculador_reagendamento.py` - Cálculo de reagendamentos
3. `executar_reagendamento.py` - Interface interativa

2 analisador_pedidos.py

2.1 Descrição

Módulo responsável pela análise de logs de produção e detecção de atividades duplicadas. Carrega logs de equipamentos e identifica atividades que aparecem em múltiplos pedidos.

2.2 Classe AnalisadorPedidos

2.2.1 Atributos

- `diretorio_logs` (str): Caminho para o diretório contendo os arquivos de log
- `pedidos` (dict): Mapeamento de (ordem, pedido) para lista de atividades
- `atividades_por_id` (defaultdict): Índice de atividades agrupadas por ID

2.2.2 Métodos Principais

`__init__(diretorio_logs)`

Inicializa o analisador de pedidos.

Parâmetros:

- `diretorio_logs` (str): Caminho para o diretório contendo os logs

`extrair_info_arquivo(nome_arquivo)`

Extraí ordem e pedido do nome do arquivo de log.

Parâmetros:

- `nome_arquivo` (str): Nome do arquivo no formato "ordem: X — pedido: Y.log"

Retorna:

- tuple: (ordem, pedido) como inteiros, ou (None, None) se formato inválido

`parse_linha_log(linha)`

Processa uma linha do log e extrai informações da atividade.

Parâmetros:

- `linha` (str): Linha do log no formato "ordem—pedido—id—produto—atividade—equipamento—inicio—fim"

Retorna:

- dict: Dicionário com dados da atividade, ou None se formato inválido

`carregar_logs()`

Carrega todos os logs do diretório especificado. Lê todos os arquivos .log no diretório, processa cada linha e indexa as atividades tanto por pedido quanto por ID de atividade.

detectar_atividades_duplicadas()

Detecta IDs de atividades que aparecem em múltiplos pedidos.

Retorna:

- dict: Mapeamento de id_atividade para lista de ocorrências

exibir_relatorio_duplicatas()

Exibe relatório formatado das atividades duplicadas.

Retorna:

- dict: Dicionário de duplicatas encontradas, ou None se não houver duplicatas

2.3 Formato dos Logs

O módulo espera logs no seguinte formato:

```
1 ordem | pedido | id_atividade | produto | atividade | equipamento | inicio | fim
```

Listing 1: Formato de linha de log

Exemplo:

```
1 1 | 1 | 10001 | Produto A | Mistura | HotMix_1 | 08:00 [01/10] | 09:30 [01/10]
```

3 `calculador_reagendamento.py`

3.1 Descrição

Módulo para cálculo de reagendamento de pedidos com atividades compartilhadas. Implementa a lógica de reagendamento backward scheduling, permitindo ajustar cronogramas de pedidos que compartilham atividades comuns.

3.2 Classe `CalculadorReagendamento`

3.2.1 Atributos

- `analisador` (`AnalisadorPedidos`): Instância do analisador de pedidos

3.2.2 Métodos Principais

`parse_horario(horario_str)`

Converte string de horário para objeto `datetime`.

Parâmetros:

- `horario_str` (`str`): Horário no formato "HH:MM [DD/MM]"

Retorna:

- `datetime`: Objeto `datetime` ou `None` se formato inválido

`formatar_horario(dt)`

Converte `datetime` para string no formato do log.

Parâmetros:

- `dt` (`datetime`): Objeto `datetime`

Retorna:

- `str`: Horário no formato "HH:MM [DD/MM]"

reagendar_pedido(...)

Reagenda um pedido usando backward scheduling. Aplica uma diferença temporal a todas as atividades do pedido, mantendo a atividade comum sincronizada com o pedido base.

Parâmetros:

- `ordem_target` (int): Ordem do pedido a reagendar
- `pedido_target` (int): Pedido a reagendar
- `horario_atividade_comum` (dict): Dados da atividade comum
- `id_atividade_comum` (int): ID da atividade compartilhada

Retorna:

- tuple: (novas_atividades, horario_final_jornada) ou None se erro

calcular_reagendamentos(duplicatas)

Calcula reagendamentos para todos os pedidos não-base.

Parâmetros:

- `duplicatas` (dict): Dicionário de atividades duplicadas

Retorna:

- tuple: (ordem_base, pedido_base, resultados)

3.3 Algoritmo de Backward Scheduling

O algoritmo funciona da seguinte forma:

1. Identifica a atividade comum entre pedidos
2. Calcula a diferença temporal entre horários original e novo
3. Aplica essa diferença a todas as atividades do pedido
4. Mantém a atividade comum sincronizada com o pedido base
5. Recalcula o horário final da jornada

```
1 horario_original_fim = parse_horario(atividade_comum['fim'])
2 horario_novo_fim = parse_horario(horario_comum_base['fim'])
3 diferenca_tempo = horario_novo_fim - horario_original_fim
4
5 # Aplicar diferenca a todas as atividades
6 novo_inicio = inicio_original + diferenca_tempo
7 novo_fim = fim_original + diferenca_tempo
```

Listing 2: Exemplo de cálculo de diferença temporal

4 executar_reagendamento.py

4.1 Descrição

Script interativo para reagendamento de pedidos com atividades duplicadas. Fornece interface de linha de comando para seleção de atividades duplicadas e cálculo de reagendamentos baseados em backward scheduling.

4.2 Funções Principais

`exibir_menu_atividades(duplicatas)`

Exibe menu numerado das atividades duplicadas para escolha do usuário.

Parâmetros:

- `duplicatas` (dict): Dicionário de atividades duplicadas

Retorna:

- list: Lista de tuplas (id_atividade, ocorrencias)

`executar_reagendamento_interativo()`

Execução interativa do reagendamento com escolha de atividades. Carrega logs, detecta duplicatas e permite ao usuário selecionar interativamente quais atividades reagendar.

4.3 Fluxo de Execução

1. Carrega logs do diretório `logs/equipamentos`
2. Valida existência de arquivos
3. Detecta atividades duplicadas
4. Exibe menu interativo de seleção
5. Calcula reagendamento para atividade selecionada
6. Exibe cronograma reagendado
7. Permite nova seleção ou saída

5 Fluxo de Dados

5.1 Diagrama de Fluxo

1. **Entrada:** Arquivos .log em logs/equipamentos/
2. **AnalisadorPedidos:** Carrega e indexa logs
3. **Deteção:** Identifica atividades duplicadas
4. **Interface:** Usuário seleciona atividade
5. **CalculadorReagendamento:** Calcula novos horários
6. **Saída:** Cronogramas reagendados

5.2 Estrutura de Dados

5.2.1 Dicionário de Pedidos

```
1 pedidos = {  
2     (ordem, pedido): [  
3         {  
4             'ordem': 1,  
5             'pedido': 1,  
6             'id_atividade': 10001,  
7             'produto': 'Produto A',  
8             'atividade': 'Mistura',  
9             'equipamento': 'HotMix_1',  
10            'inicio': '08:00 [01/10]',  
11            'fim': '09:30 [01/10]',  
12        },  
13        ...  
14    ]  
15 }
```

5.2.2 Dicionário de Duplicatas

```
1 duplicatas = {  
2     id_atividade: [  
3         (ordem, pedido, dados_atividade),  
4         (ordem, pedido, dados_atividade),  
5         ...  
6     ]  
7 }
```

6 Exemplo de Uso

6.1 Uso Programático

```
1 from analisador.analisador_pedidos import AnalisadorPedidos
2 from analisador.calculador_reagendamento import CalculadorReagendamento
3
4 # Carregar logs
5 analisador = AnalisadorPedidos("logs/equipamentos")
6 analisador.carregar_logs()
7
8 # Detectar duplicatas
9 duplicatas = analisador.detectar_atividades_duplicadas()
10
11 # Calcular reagendamento
12 calculador = CalculadorReagendamento(analisador)
13 ordem_base, pedido_base, resultados = calculador.calcular_reagendamentos(
14     duplicatas)
15
16 # Exibir resultados
17 for (ordem, pedido), dados in resultados.items():
18     calculador.exibir_cronograma_reagendado(ordem, pedido, dados['atividades'])
```

Listing 3: Exemplo de uso das classes

6.2 Uso via Interface

```
1 $ python analisador/executar_reagendamento.py
```

Listing 4: Execução do script interativo

7 Padrões e Convenções

7.1 Documentação

O módulo segue o padrão **Google Docstrings**, recomendado pela comunidade Python e PEP 257:

```
1 def parse_horario(self, horario_str):
2     """
3     Converte string de horario para objeto datetime.
4
5     Args:
6         horario_str (str): Horario no formato "HH:MM [DD/MM]"
7
8     Returns:
9         datetime: Objeto datetime ou None se formato invalido
10    """
11    ...
```

Listing 5: Exemplo de docstring

7.2 Nomenclatura

- Classes: PascalCase (ex: `AnalisadorPedidos`)
- Funções/métodos: snake_case (ex: `carregar_logs`)
- Constantes: UPPER_CASE (quando aplicável)
- Variáveis privadas: prefixo `_` (quando aplicável)

7.3 Estrutura de Código

- Docstring de módulo no topo do arquivo
- Imports organizados (stdlib, third-party, local)
- Classes e funções documentadas
- Type hints quando apropriado

8 Manutenção e Extensão

8.1 Adicionar Novos Formatos de Log

Para adicionar suporte a novos formatos de log, modifique o método `parse_linha_log()` em `AnalisadorPedidos`.

8.2 Algoritmos Alternativos

Para implementar algoritmos de reagendamento alternativos ao backward scheduling, crie novos métodos na classe `CalculadorReagendamento`.

8.3 Validações Adicionais

Adicione validações no método `carregar_logs()` para verificar integridade dos dados.

9 Dependências

9.1 Bibliotecas Python

- `datetime` - Manipulação de datas e horários
- `collections.defaultdict` - Estruturas de dados
- `re` - Expressões regulares para parsing
- `os, sys` - Operações de sistema

9.2 Módulos Internos

- `analisador_pedidos` → `calculador_reagendamento`
- `analisador_pedidos, calculador_reagendamento` → `executar_reagendamento`

10 Conclusão

O módulo `analisador` fornece uma solução completa para análise de logs de produção, detecção de conflitos de recursos e cálculo de reagendamentos. A arquitetura modular permite fácil manutenção e extensão, enquanto a interface interativa facilita o uso por operadores.